src/figures.py

Every README figure, regenerated from the saved results

No figure in this project is drawn by hand or lives only in a notebook cell. This module rebuilds each one from the evaluation results, so a figure can never drift from the numbers it claims to show. run_eval.py calls these after computing results.

Walkthrough of all 100 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Every figure in the README, regenerated from the evaluation results.
 2
 3  No figure in this project is drawn by hand or lives only in a notebook cell — each
 4  one is produced here from the saved results, so it is always reproducible and can
 5  never drift from the numbers it claims to show. `run_eval.py` calls these after
 6  computing the results.
 7  """
```

Lines 1 to 7 are the module docstring, stating the principle: every figure is produced here from saved results, so it is always reproducible and never drifts, and run_eval.py is the caller.

```
 8
 9  from __future__ import annotations
10
11  import pandas as pd
12
13  from . import config
```

Lines 8 to 13 are imports: postponed annotations (line 9), pandas for shaping the results into frames (line 11), and the project config for output paths (line 13). Matplotlib is deliberately not imported here — see the lazy loader below.

```
14
15  # A calm, consistent palette so the project's figures read as one set.
16  _NAIVE_COLOR = "#9aa5b1"
17  _AGENT_COLOR = "#2f6f4f"
```

Lines 14 to 17 define the palette: _NAIVE_COLOR (a muted grey) and _AGENT_COLOR (the project green), so naive and agent bars are colored consistently across every figure and the set reads as one visual family.

```
18
19
20  def _lazy_plt():
21      """Import matplotlib lazily and apply a clean default style."""
22      import matplotlib
23
24      matplotlib.use("Agg")  # headless: write files, never open a window
25      import matplotlib.pyplot as plt
26
27      plt.rcParams.update({"figure.dpi": 120, "font.size": 10, "axes.spines.top": False,
28                           "axes.spines.right": False})
29      return plt
```

Lines 18 to 29 define _lazy_plt, which imports matplotlib only when a figure is actually drawn. Line 24 sets the Agg backend (headless: write files, never open a window), important for running under a script or notebook execution. Lines 27 to 28 apply a clean default style (DPI, font size, and hiding the top and right spines). Line 29 returns the pyplot module for the caller to use.

```
30
31
32  def retrieval_metrics_figure(retrieval_rows: list[dict]) -> None:
33      """Bar chart of hit@k / recall@k / MRR, overall and by hop type."""
34      plt = _lazy_plt()
35      df = pd.DataFrame(retrieval_rows)
36      metrics = ["hit@k", "recall@k", "mrr"]
37
38      groups = {"all": df, "single-hop": df[df.hop == "single"], "multi-hop": df[df.hop == "multi"]}
39      fig, ax = plt.subplots(figsize=(7, 4))
40      x = range(len(metrics))
41      width = 0.25
42      for i, (label, sub) in enumerate(groups.items()):
43          vals = [sub[m].mean() if len(sub) else 0.0 for m in metrics]
44          ax.bar([xi + i * width for xi in x], vals, width, label=label)
45      ax.set_xticks([xi + width for xi in x])
46      ax.set_xticklabels(metrics)
47      ax.set_ylim(0, 1)
48      ax.set_ylabel("score")
49      ax.set_title("Retrieval quality (higher is better)")
50      ax.legend()
51      fig.tight_layout()
52      fig.savefig(config.FIGURES_DIR / "01_retrieval_metrics.png")
53      plt.close(fig)
```

Lines 30 to 53 define retrieval_metrics_figure — a grouped bar chart of hit@k, recall@k, and MRR. It loads pyplot lazily (line 34), frames the rows (line 35), and names the three metrics (line 36). Line 38 splits the data into all, single-hop, and multi-hop groups. Lines 39 to 44 draw one bar cluster per group, guarding empty groups with a 0.0 fallback (line 43). Lines 45 to 50 set the tick labels, the 0-to-1 y-limit, axis label, title, and legend. Lines 51 to 53 tighten the layout, save the PNG to the figures directory, and close the figure to free memory.

```
54
55
56  def answer_quality_figure(answer_rows: list[dict]) -> None:
57      """Naive vs agent, groundedness + correctness, split by hop type."""
58      plt = _lazy_plt()
59      df = pd.DataFrame(answer_rows)
60
61      fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
62      for ax, metric in zip(axes, ["groundedness", "correctness"]):
63          pivot = df.groupby(["hop", "system"])[metric].mean().unstack("system")
64          pivot = pivot.reindex(columns=["naive", "agent"])
65          pivot.plot(kind="bar", ax=ax, color=[_NAIVE_COLOR, _AGENT_COLOR], rot=0,
66                     legend=(metric == "groundedness"))
67          ax.set_title(metric)
68          ax.set_xlabel("question type")
69          ax.set_ylim(0, 5)
70      axes[0].set_ylabel("judge score (1-5)")
71      fig.suptitle("Answer quality: naive RAG vs agentic RAG")
72      fig.tight_layout()
73      fig.savefig(config.FIGURES_DIR / "02_answer_quality.png")
74      plt.close(fig)
```

Lines 54 to 74 define answer_quality_figure — naive versus agent, for groundedness and correctness, split by hop type. It frames the answer rows (line 59) and makes two side by side subplots sharing a y-axis (line 61). For each metric (line 62) it pivots the mean judge score by hop and system (line 63), orders the columns naive then agent (line 64), and draws a grouped bar with the palette, showing the legend only once (lines 65 to 66). Lines 67 to 70 set titles, x-labels, the 0-to-5 range, and the shared y-label. Lines 71 to 74 add the overall title, tighten, save, and close.

```
75
76
77  def cost_tradeoff_figure(answer_rows: list[dict]) -> None:
78      """The other side of the ledger: how many retrievals each system spends."""
79      plt = _lazy_plt()
80      df = pd.DataFrame(answer_rows)
81      means = df.groupby("system")["n_retrievals"].mean().reindex(["naive", "agent"])
82
83      fig, ax = plt.subplots(figsize=(5, 4))
84      ax.bar(means.index, means.values, color=[_NAIVE_COLOR, _AGENT_COLOR])
85      ax.set_ylabel("avg. retrieval calls per question")
86      ax.set_title("Cost of the agent's iterative retrieval")
87      for i, v in enumerate(means.values):
88          ax.text(i, v, f"{v:.1f}", ha="center", va="bottom")
89      fig.tight_layout()
90      fig.savefig(config.FIGURES_DIR / "03_cost_tradeoff.png")
91      plt.close(fig)
```

Lines 75 to 91 define cost_tradeoff_figure — the other side of the ledger: how many retrievals each system spends per question. It frames the rows (line 80) and computes the mean n_retrievals per system, ordered naive then agent (line 81). Lines 83 to 84 draw the two bars in the palette. Lines 85 to 86 set the y-label and title. Lines 87 to 88 annotate each bar with its value to one decimal. Lines 89 to 91 tighten, save the PNG, and close.

```
 92
 93
 94  def all_figures(retrieval_rows: list[dict], answer_rows: list[dict]) -> None:
 95      """Regenerate every README figure. Called by run_eval.py."""
 96      config.ensure_dirs()
 97      retrieval_metrics_figure(retrieval_rows)
 98      if answer_rows:
 99          answer_quality_figure(answer_rows)
100          cost_tradeoff_figure(answer_rows)
```

Lines 92 to 100 define all_figures, the single entry point run_eval.py calls. It ensures the output directories exist (line 96), always draws the retrieval-metrics figure (line 97, which needs no API key), and — only if answer rows are present (line 98) — draws the answer-quality and cost-tradeoff figures (lines 99 to 100). So the local retrieval figure always regenerates, while the key-gated answer figures render only once the answer evaluation has produced data.